МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КАРПАТСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ВАСИЛЯ СТЕФАНИКА
ФАКУЛЬТЕТ МАТЕМАТИКИ ТА ІНФОРМАТИКИ

ЛАБОРАТОРНА РОБОТА 6–7

з дисципліни Професійна практика програмної інженерії

Розробка фронтенду застосунку для планування туристичних маршрутів із використанням геолокації

Виконав студент групи ІПЗ-42

Ницак Олег

Перевірив: Пікуляк Микола Васильович

Івано-Франківськ 2026

Мета роботи та постановка задачі

У лабораторних роботах 6–7 розроблено користувацький інтерфейс Trailway для планування туристичних маршрутів. Реалізація охоплює адаптивні екрани, навігацію, форми з перевіркою даних, відображення карти, асинхронні запити до власного REST API та збереження планів без мережі. Навчальний Node.js сервер і вбудований PostgreSQL PGlite дозволяють виконувати CRUD над постійними даними без окремого сервера бази даних.

Робота продовжує предметну область попередніх лабораторних: користувач формує маршрут із туристичних локацій або власних координат, зберігає його та проходить як окрему подорож. Сутності Route, RoutePoint, Trip і TrackPoint зберігають різні життєві цикли плану та фактичного проходження. Каталог містить природні локації, оглядові майданчики й музеї Івано-Франківська та околиць.

Для виконання вебвимог лабораторної створено React-клієнт із Vite та режимом PWA. Він працює в настільному й мобільному браузері. Архітектуру попереднього мобільного етапу адаптовано до вебсередовища: контракт Node.js API і модель PostgreSQL збережено, а локальні дані клієнта розміщено в IndexedDB. Застосунок можна відкривати як звичайну сторінку; нативна Android або iOS збірка не є результатом цих лабораторних.

Основний сценарій починається з перегляду каталогу, переходить до редактора й завершується збереженням плану. Гість може працювати локально. Після входу його плани переносяться до особистого локального простору й надсилаються на сервер під час синхронізації. Авторизований турист має доступ до своїх маршрутів і подорожей. Адміністратор додатково керує каталогом локацій, але не отримує доступу до приватних маршрутів інших користувачів.

Готовий маршрут містить щонайменше дві точки; чернетка може містити одну. Максимальна кількість точок дорівнює двадцяти. Інтерфейс показує приблизну довжину плану, окремо записує фактичний GPS-трек та повідомляє про стан мережі, збереження й конфлікти даних.

Карта навігації та переходи



Рисунок 6.1 — Карта навігації Trailway

Головна сторінка поєднує список маршрутів і перегляд вибраного плану. Редактор відкривається модально, тому після збереження користувач залишається в контексті списку. Каталог дозволяє створити план від вибраної локації. Після початку проходження інтерфейс переходить до екрана подорожей.

Навігація використовує адреси виду /#/routes, /#/places, /#/trips та /#/account. Сторінка /#/admin доступна ролі адміністратора. Поширений маршрут відкривається за /#/share/token без облікового запису. На вузькому екрані ті самі розділи доступні через нижню панель навігації.

Wireframe основного екрана

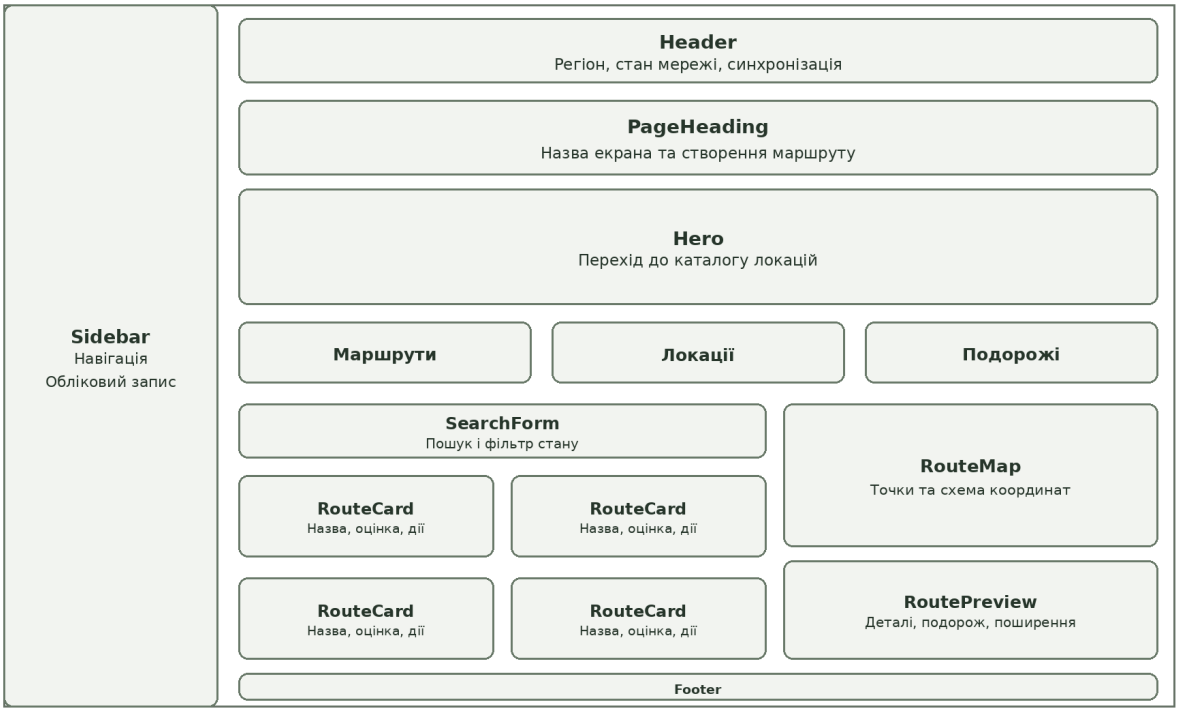


Рисунок 6.2 — Wireframe списку маршрутів і перегляду карти

Макет визначає стабільну структуру: навігація ліворуч, стан системи зверху, основна дія створення маршруту біля заголовка, а список і карта поруч. Картка містить назву, опис, оцінки та кнопки редагування й видалення. Праворуч показано точки вибраного маршруту, початок подорожі та поширення.

На смартфоні основні блоки розташовуються послідовно. Картки займають усю доступну ширину, карта переходить нижче списку, а меню — до нижньої панелі. Прототип переходів реалізований у самому React-застосунку: кнопки відкривають редактор, діалог входу, каталог і екран GPS без перезавантаження всього інтерфейсу.

Декомпозиція інтерфейсу

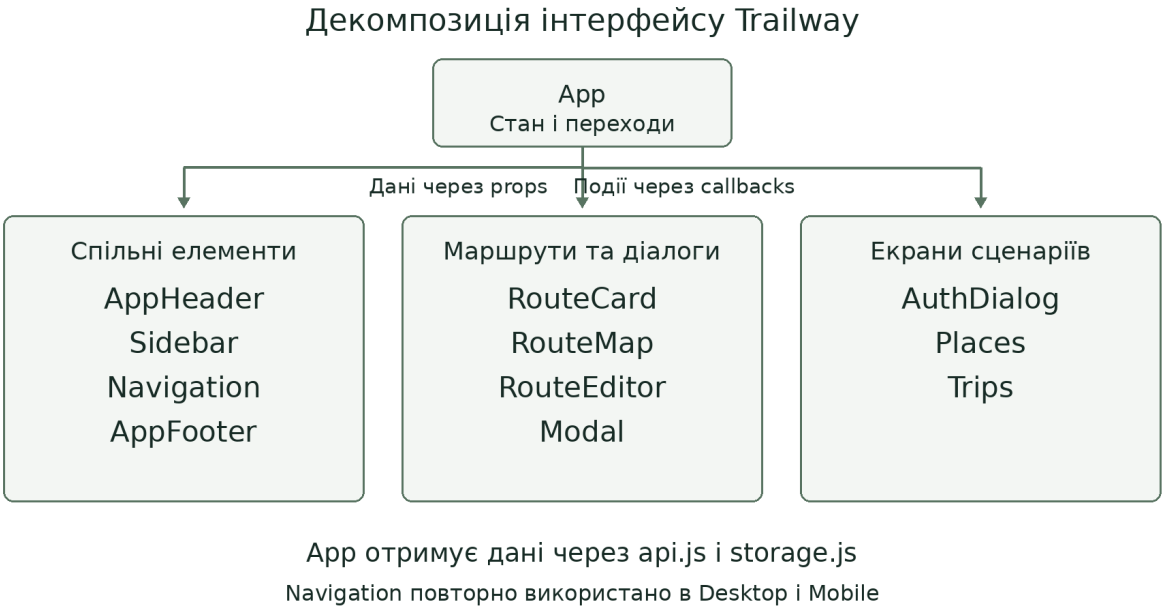


Рисунок 6.3 — Декомпозиція клієнта на компоненти

App керує спільним станом і переходами. AppHeader відображає стан мережі, Sidebar містить бічну панель, Navigation використовує ті самі переходи на комп’ютері й телефоні, а AppFooter завершує сторінку. RouteCard представляє план, RouteMap показує координати, Modal керує діалогом і фокусом. RouteEditor, AuthDialog, Places та Trips реалізують предметні сценарії. Роботу з HTTP і локальними даними винесено в api.js та storage.js.

Файл або каталог	Відповідальність
client/src/App.jsx	Навігація, загальний стан і збереження
client/src/components/	Шапка, бічна панель, навігація, футер, карта, картка та діалоги
client/src/pages/	Редактор, вхід, каталог і подорожі
client/src/api.js	Асинхронні HTTP-запити та відновлення сесії
client/src/storage.js	Локальні маршрути й черга синхронізації
client/src/styles.css	Сітки, адаптивність і стани інтерфейсу
client/public/sw.js	Кешування оболонки PWA

Структура каталогів та файлів

Клієнт і сервер розміщено в одному npm-проєкті. Дерево показує вихідні файли, SQL-схему, тести та документацію. Команди запуску виконуються з кореня trailway.

Структура вихідних файлів

```
trailway/
├── package.json  ·  package-lock.json
├── README.md  ·  start-windows.cmd
├── client/
│   ├── index.html  ·  vite.config.js
│   ├── public/
│   │   └── icon.svg  ·  manifest.webmanifest  ·  sw.js
│   └── src/
│       ├── main.jsx  ·  App.jsx  ·  styles.css
│       ├── api.js  ·  storage.js
│       ├── components/
│       │   ├── AppHeader.jsx  ·  AppFooter.jsx
│       │   ├── Sidebar.jsx  ·  Navigation.jsx
│       │   ├── RouteCard.jsx  ·  RouteMap.jsx
│       │   └── Modal.jsx  ·  Icon.jsx
│       └── pages/
│           ├── AuthDialog.jsx  ·  RouteEditor.jsx
│           └── Places.jsx  ·  Trips.jsx
├── server/
│   ├── sql/001_schema.sql
│   └── src/
│       ├── app.js  ·  index.js  ·  setup.js  ·  db.js
│       ├── routes-service.js  ·  trips-service.js
│       ├── security.js  ·  validation.js  ·  seed.js
│       └── errors.js  ·  openapi.js
├── scripts/
│   └── dev.js  ·  check.js  ·  stamp-sw.js
├── tests/
│   ├── api.test.js  ·  client-api.test.js
│   └── persistence.test.js  ·  e2e.mjs
└── docs/
    └── diagrams/  ·  evidence/  ·  reports/
```

Рисунок 6.4 — Дерево файлів fullstack-проєкту

App.jsx керує екранами й даними. components містить спільні елементи інтерфейсу, а pages — форми та предметні екрани. api.js обробляє HTTP-запити й сесію; storage.js керує IndexedDB. Серверний каталог src містить контролери, перевірки та бізнес-логіку, а sql — міграцію PostgreSQL.

Готовий інтерфейс на комп’ютері

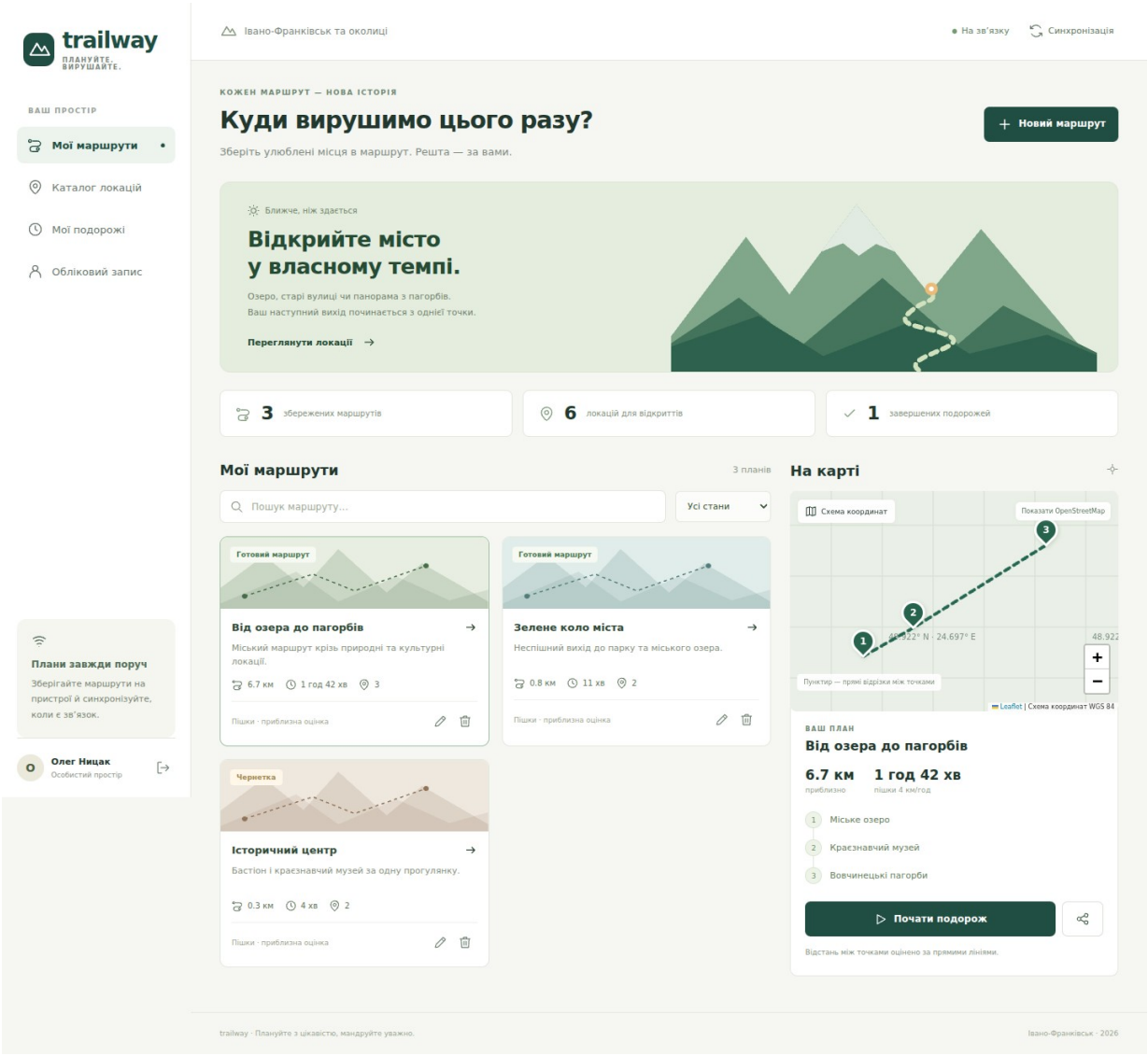


Рисунок 6.5 — Головний екран у браузері шириною 1440 px

Скриншот отримано з працюючого React-клієнта після входу туриста. На головному екрані відображено три початкові маршрути, шість локацій і одну демонстраційну завершену подорож. Пошук та фільтр стану звужують список; за наявності понад шести планів доступна пагінація карток.

Пунктир на карті показує прямі відрізки між точками. Показники плану підписано як приблизні, оскільки реалізація оцінює відстань за координатами й не розраховує шлях дорогами. Для перегляду реальної картографічної основи користувач може ввімкнути онлайн-тайли OpenStreetMap.

Адаптивність мобільного інтерфейсу

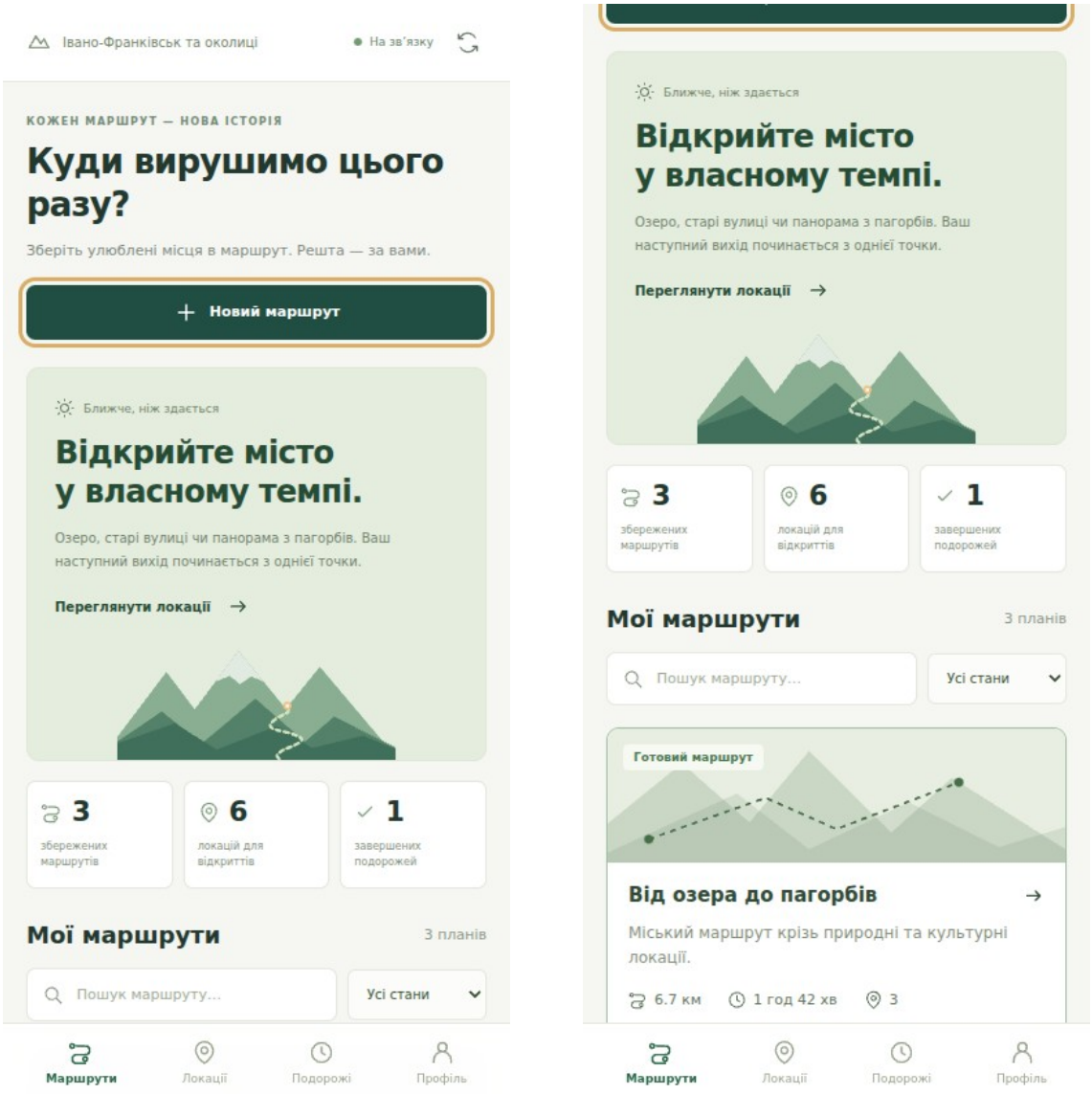


Рисунок 6.6 — Мобільний екран та картки маршрутів при ширині 390 px

Мобільний макет перевірено у viewport 390 × 844 px. Під час браузерного тесту ширина документа не перевищила ширину viewport. Заголовок переноситься, кнопка створення займає весь рядок, статистика залишається компактною, а картки доступні в одному стовпці.

Нижня навігація забезпечує прямий доступ до основних розділів. Для кнопок, модальних вікон і повідомлень залишено місце над нею. CSS Grid і Flexbox змінюють розташування блоків у media queries, а ResizeObserver оновлює розміри карти після зміни ширини контейнера.

Редактор та перевірка даних

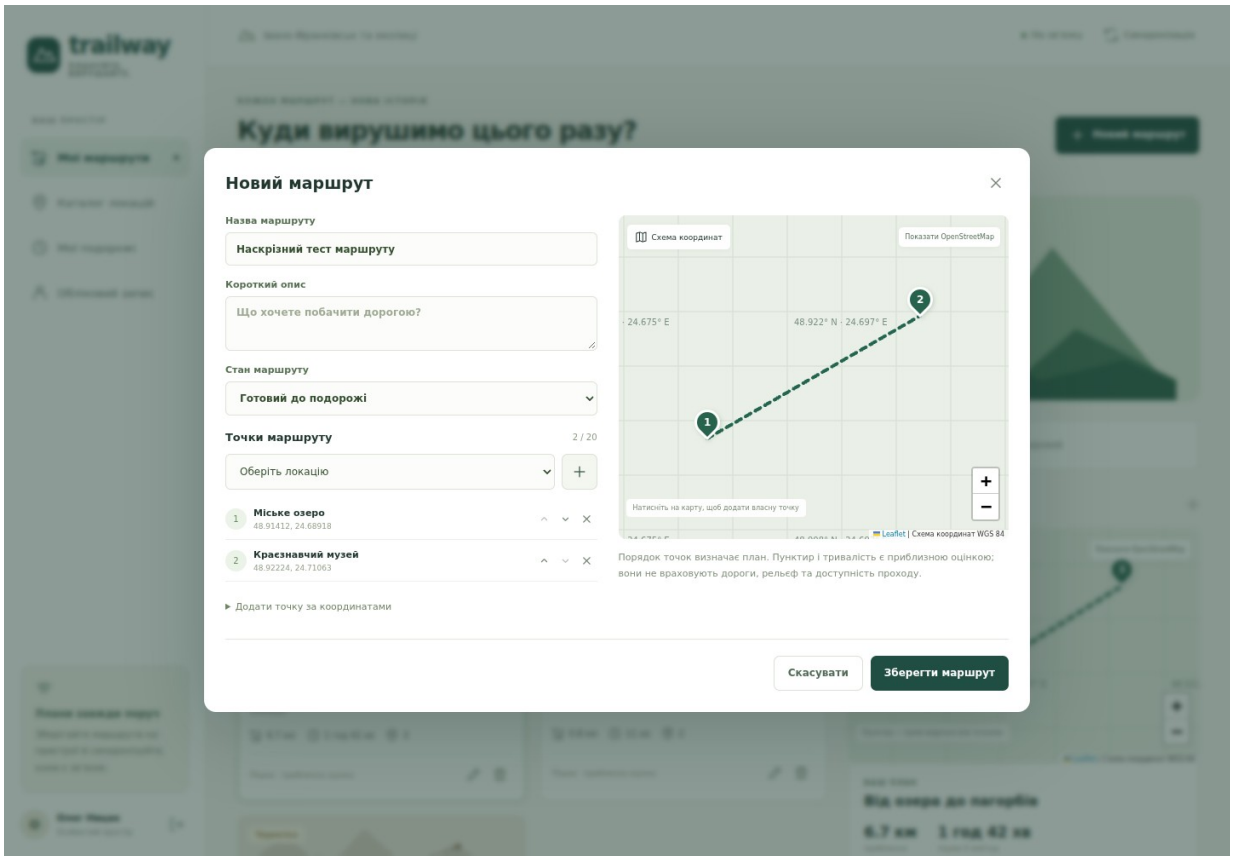


Рисунок 6.7 — Форма створення маршруту з двома точками

Редактор перевіряє назву, кількість точок та допустимість готового стану. Користувач додає точки з каталогу, вводить координати або натискає на карту. Кнопки підняття й опускання змінюють порядок; видалення точки одразу оновлює геометрію. Незбережені зміни можна скасувати, не змінюючи список.

Поля мають видимі підписи, обмеження довжини та числові діапазони. Стан busy вимикає повторне збереження. Повідомлення про помилку виводиться через role=alert. Сервер повторює перевірки, тому обходження HTML-обмежень не дозволяє записати некоректні дані.

Каталог локацій та прототип сценаріїв

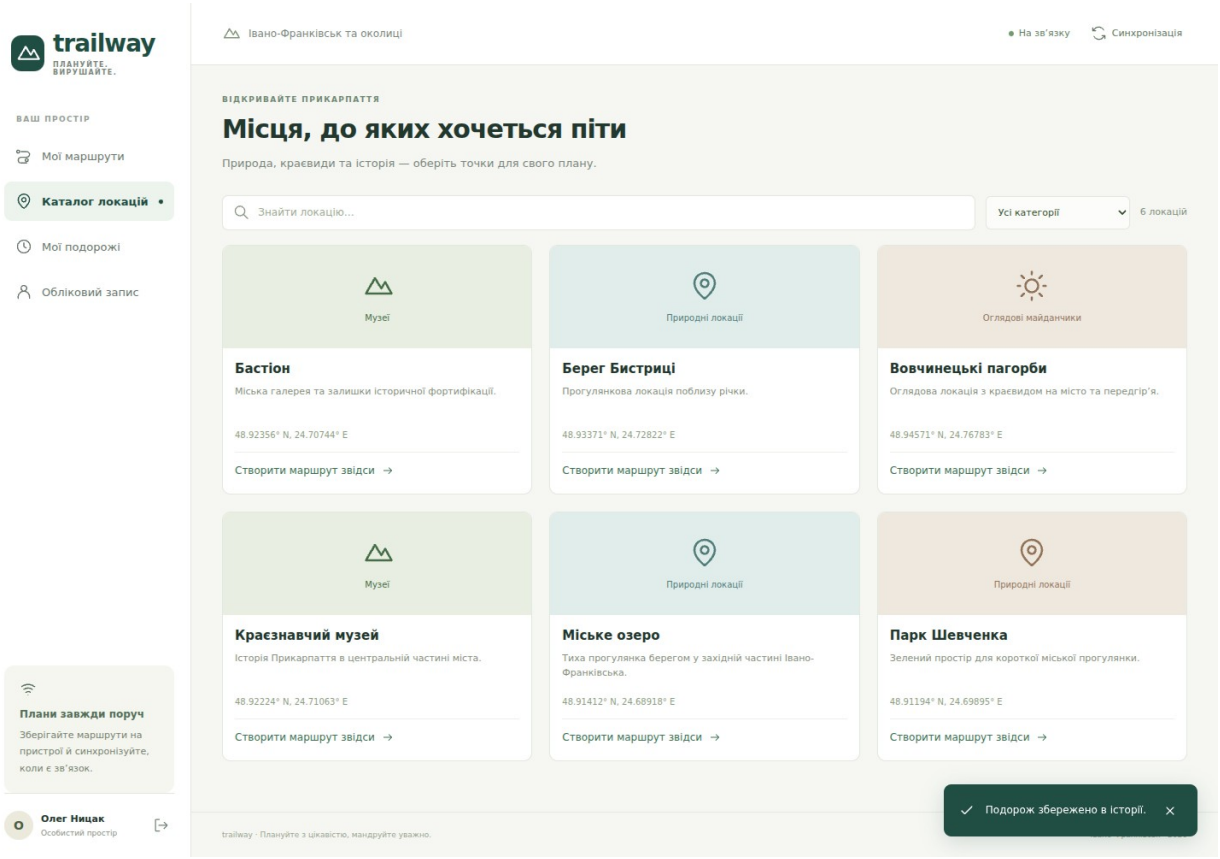


Рисунок 6.8 — Каталог локацій у працюючому React застосунку

Каталог відображає назву, категорію, опис і координати локації. Пошук і фільтр категорії дають змогу звузити набір даних. Вибір локації відкриває редактор із початковою точкою, після чого користувач додає наступні місця та зберігає чернетку або готовий план.

Інтерактивний прототип переходів реалізовано в самому застосунку. Навігація веде від списку маршрутів до каталогу, модального редактора, облікового запису й історії подорожей. Дія «Почати подорож» відкриває GPS екран; пауза, продовження та завершення змінюють стан проходження, а вихід із редактора повертає користувача до попереднього екрана.

План і фактичний трек показуються окремо. Карта на ілюстраціях є схемою координат WGS 84; онлайн основу OpenStreetMap користувач вмикає окремою дією. Фізичний GPS телефона та запис у фоні потребують додаткової перевірки на пристрої.

Код основних React компонентів

AuthDialog зберігає значення полів і стан виконання у useState. Подія submit надсилає облікові дані через API; у разі успіху onSuccess оновлює користувача в App, а помилка залишається видимою в діалозі. Нижче наведено фрагмент обробника з client/src/pages/AuthDialog.jsx у розгорнутому форматуванні.

```
// Значення форми та стан виконання належать компоненту.
const [email, setEmail] = useState('');
const [password, setPassword] = useState('');
const [busy, setBusy] = useState(false);
const [error, setError] = useState('');
const submit = async e => {
  e.preventDefault();
  setBusy(true);
  setError('');
  try {
    const path = register ? '/auth/register' : '/auth/login';
    const body = {email, password,
      ...(register ? {display_name: name} : {})};
    const session = await api(path, {method: 'POST', body});
    setToken(session.access_token);
    await onSuccess(session.user); // App приймає успішний вхід.
    onClose();
  } catch (e) {
    setError(e.message);
  } finally {
    setBusy(false);
  }
};
```

RouteCard отримує маршрут та функцію onSelect через props. Подія вибору картки змінює selected у App; RouteMap отримує точки вже вибраного плану. Такий обмін не потребує глобальної зміни DOM або дублювання даних картки в самій карті.

```
// Фрагмент JSX компонента RouteCard.
<button className="card-title" onClick={() => onSelect(route)}>
  {route.name}
  <Icon name="arrow" size={17} />
</button>
```

Стан та асинхронна взаємодія з API

React useState зберігає користувача, поточний екран, маршрути та стани діалогів. Успішний запис оновлює локальний кеш і картки. При втраті мережі зміни записуються до IndexedDB. API-клієнт координує відновлення сесії, а номер її покоління відхиляє відповідь, яка надійшла після зміни акаунта. Нижче наведено функцію refreshSession у компактному форматуванні.

```
export async function refreshSession() {
  if (authTransitions) throw new SessionChangedError();
  if (!refreshPromise) {
    const generation = sessionGeneration;
    refreshPromise = fetch('/api/auth/refresh', {
      method: 'POST',
      credentials: 'same-origin',
      headers: {'Content-Type': 'application/json'},
      body: '{}'
    }).then(decode).then(session => {
      if (generation !== sessionGeneration) {
        throw new SessionChangedError();
      }
      accessToken = session.access_token;
      return session;
    }).finally(() => {
      refreshPromise = null;
    });
  }
  return refreshPromise;
}
```

Запити використовують fetch і відносний шлях /api. Access-token передається з пам’яті сторінки в Authorization. Після 401 клієнт один раз відновлює сесію через HttpOnly cookie та повторює запит. Спільний refreshPromise об’єднує одночасні оновлення. Вхід і вихід очікують попередній refresh, щоб його пізня відповідь не перезаписала cookie нової сесії.

Після зміни користувача App очищує приватний екран і перевіряє поточний scope перед кожним оновленням стану. Запізніла приватна відповідь не відображається в гостьовому режимі. Відповідь 204 обробляється без читання JSON. Vite проксіює /api у розробленні, а зібраний Express-застосунок віддає UI та API з одного origin.

CRUD операції та контракт REST API

Клієнт надсилає JSON через fetch і отримує відповідь із фактичним серверним станом. Інтерфейс оновлює кеш після успішного запису. Приватні маршрути доступні лише власникові; номер базової версії дозволяє виявити конкурентну зміну.

Метод і адреса	Результат	Дія інтерфейсу
GET /api/routes	200 items і total	Читання списку
GET /api/routes/:id	200 маршрут із точками	Деталі плану
POST /api/routes	201 створений маршрут	Створення
PUT /api/routes/:id	200 нова версія	Редагування
DELETE /api/routes/:id	204 без тіла	Видалення
POST /api/sync	200 результати операцій	Надсилання черги

Фрагмент онлайн гілки saveRoute в App.jsx показує вибір POST або PUT, асинхронний запис і оновлення локального стану. scopedApi та ensureScope відхиляють результат, якщо користувач змінився під час запиту.

```
// Сервер повертає збережену версію маршруту.
const saved = await scopedApi(
  isNew ? "/routes" : `/routes/${route.id}`,
  {
    method: isNew ? "POST" : "PUT",
    body: { ...route, base_version: route.version },
  },
);
ensureScope();
await local.mutate(scope, (data) => {
  data.routes = [
    saved,
    ...data.routes.filter((r) => r.id !== route.id),
  ];
});
await refreshLocal();
ensureScope();
setSelected(route.id);
```

Помилка валідації 400 залишається видимою у формі, 401 завершує або відновлює сесію, 409 означає конфлікт версій. Мережева помилка дозволяє зберегти зміну локально. Відповідь 204 обробляється без читання JSON.

Локальні дані та синхронізація

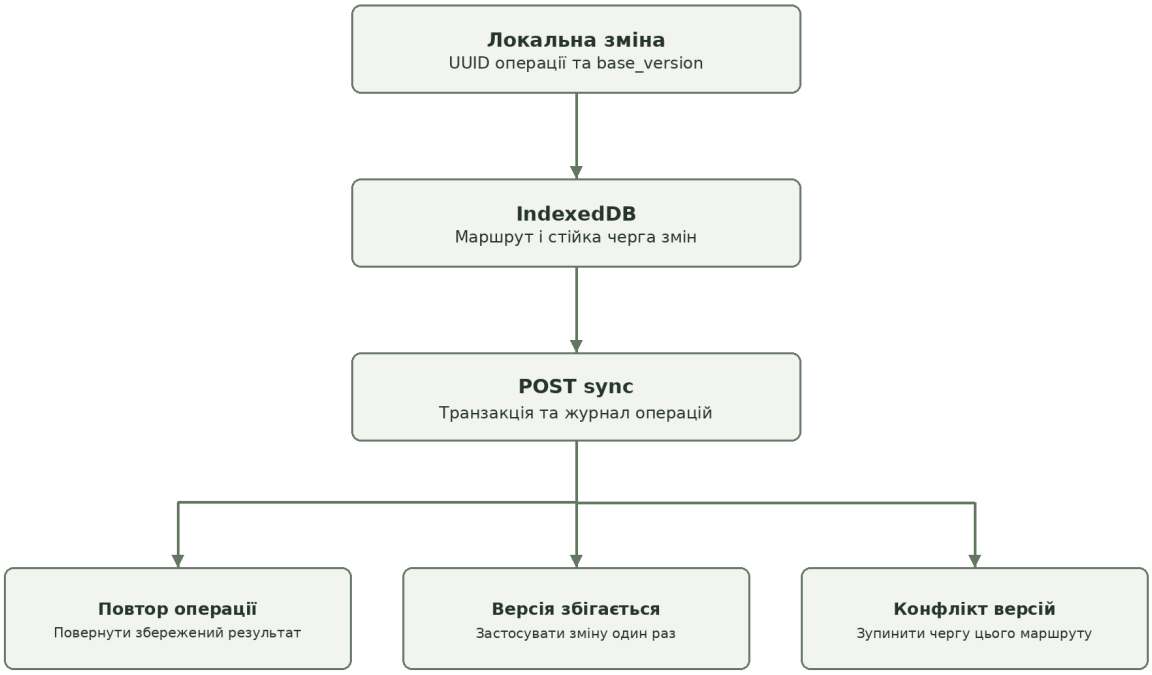


Рисунок 6.9 — Оброблення локальних змін і конфліктів

IndexedDB містить окремий простір гостя та окремі простори користувачів. Черга зберігає client_operation_id, entity_id, operation_type, payload і base_version. Ідентифікатор операції не змінюється при повторному надсиланні. Після успішної відповіді операція вилучається з черги, а актуальний маршрут читається із сервера.

Конфлікт зупиняє також наступні зміни цього самого маршруту. Це не дозволяє пізнішій локальній операції перезаписати серверну копію після невдалого попереднього оновлення. Інтерфейс пропонує явну дію відкидання локальних змін та завантаження серверного стану.

Service worker попередньо кешує HTML, JS, CSS та піктограму, тому вже відкритий зібраний застосунок можна перезавантажити без мережі. Приватні API-відповіді не кешуються service worker. Кеш каталогу, маршрутів і GPS-черга керуються окремо через IndexedDB.

Обґрунтування проєктних рішень

React обрано для повторного використання компонентів і узгодження стану списку, карти та редактора. App володіє спільними даними, а форма зберігає свою чернетку в useState. Передавання props і callbacks робить відповідальність компонентів явною та дозволяє скасувати редагування без зміни збереженого плану.

Vite забезпечує запуск клієнта в режимі розроблення та створення production збірки. Відносна адреса /api й проксі прибирають потребу в окремих налаштуваннях origin для кожного компонента. Навчальний сервер із PGlite дозволяє перевірити REST операції зі збереженням записів після перезапуску.

CSS Grid керує сітками карток і списку з картою, Flexbox розміщує шапку та навігацію. При ширині до 900 px список і карта розташовуються вертикально. При ширині до 680 px бічна панель приховується, робоча область займає всю ширину, а навігація переходить до нижнього краю. Фрагмент styles.css показує основні правила цього переходу.

```
@media (max-width: 680px) {
  .sidebar { display: none; }
  .workspace { margin: 0; width: 100%; }
  .mobile-nav {
    display: flex;
    position: fixed;
    bottom: 0; left: 0; right: 0;
  }
}
```

Навігація використовує hash адреси, які можна відкривати без додаткового налаштування серверного маршрутизатора SPA. Один компонент Navigation забезпечує однаковий набір переходів для комп’ютера й телефона. Текстова ознака стану, індикація вибраного розділу та повідомлення role=alert або role=status дозволяють зрозуміти результат дії.

Геолокація запитується лише після дії користувача. Приблизний план за координатами не використовується як обіцянка прохідного шляху дорогами. Локальні плани й черга змін зберігаються в IndexedDB, а service worker кешує оболонку застосунку. Це підтримує перегляд і планування без мережі; початок і завершення серверної подорожі потребують зв’язку.

Перевірка інтерфейсу та usability

Під час перевірки 11 жовтня 2026 року пройдено 37 автоматичних тестів API, клієнтських запитів і постійності даних та 16 браузерних сценаріїв із реальним Express API й PostgreSQL PGlite. Production-збірка Vite завершилась успішно. CRUD перевірено за фактичними записами й версіями в базі; запізнілі відповіді після виходу та переходу між посиланнями не змінювали поточний приватний екран.

Перевірка	Підтверджений результат
Настільний екран	Вхід і відображення початкових даних
Мобільний екран	390 px без горизонтального переповнення
CRUD маршруту	Створення, редагування та видалення через UI
Поширення	Перегляд, відкликання та перехід між посиланнями
GPS-сценарій	Тестова координата, пауза, продовження, завершення
Офлайн-режим	Збереження, перезавантаження та повторне підключення
Конфлікт версій	Блокування наступних локальних змін
Гостьові плани	Імпорт та ізоляція після виходу із запізнілою відповіддю
Панель адміністратора	Запис локації до PostgreSQL
Помилки JavaScript	Не зафіксовано в завершеному сценарії
Клавіатурне керування	Tab і Shift+Tab утримують фокус, Escape скасовує форму
Валідація готового плану	Одна точка викликає помилку без запису до бази
Мобільна навігація	Каталог відкривається, активний пункт має aria-current

Usability забезпечують видимі назви дій, послідовні картки, стан мережі, порожні екрани та повідомлення. Видалення потребує підтвердження, редактор дозволяє скасування. Modal підтримує Tab, Shift+Tab, Escape і повернення фокусу до кнопки відкриття; приховані поля закритого details не входять до обходу. Navigation позначає поточний розділ через aria-current. За prefers-reduced-motion вимкнено зайві переходи.

Оцінка usability має характер перевірки реалізованих сценаріїв. Окреме дослідження з учасниками не проводилося. GPS перевірено браузерною тестовою координатою; поведінка фізичного телефона та фонового запису потребує перевірки на пристрої.

Запуск та висновки

Проєкт запускається з папки trailway, що містить package.json. Потрібен Node.js 22.13 або новіший. Після встановлення npm-залежностей сервер автоматично підключає локальну PostgreSQL PGlite, застосовує міграцію й додає демонстраційні записи під час першого запуску. Команди перевірено в робочому середовищі; стандартний запуск не потребує Docker чи додаткових служб.

```
npm ci
npm run setup
npm run build
npm start

# Інтерфейс: http://localhost:3001
# Swagger:   http://localhost:3001/docs
```

Для перевірки можна ввійти як oleh@example.com із паролем Tourist2026! або як admin@example.com із паролем Admin2026!. У розробленні npm run dev одночасно запускає Vite на порту 5173 та API на порту 3001. Докладні інструкції й умови геолокації наведено в README архіву.

У лабораторних роботах 6–7 створено компонентний адаптивний фронтенд, реалізовано навігацію та форми, асинхронний CRUD через API, авторизаційні діалоги, локальне збереження й синхронізацію. Інтерфейс інтегровано з власним сервером та перевірено в настільному й мобільному viewport. Результат забезпечує повний навчальний сценарій від планування маршруту до збереження подорожі.

Використані матеріали

Умова лабораторної роботи №6–7 з прикріпленого документа Lab 6-7(1).docx. Звіт Lab5.pdf використано як зразок титульної сторінки й моделі даних.

Бесіда з попередніми лабораторними: https://chatgpt.com/share/6aca5563-35f4-83eb-a7c5-40e2290a6f08

React: https://react.dev/learn

Vite: https://vite.dev/guide/

Leaflet: https://leafletjs.com/reference.html
PGlite: https://pglite.dev/docs/about
Геолокація: https://developer.mozilla.org/en-US/docs/Web/API/Geolocation_API